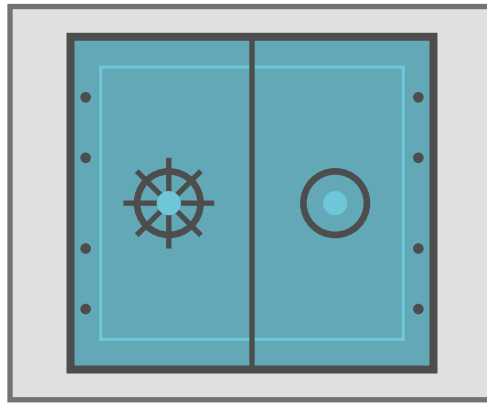


Criptografía y Seguridad

Seguridad en aplicaciones

Flujo de Información y Malware



Apunte integral — Teoría + Práctica

Clase 9 · Capítulos 16–17, *Computer Security: Art and Science* (M. Bishop)

Práctica: *CWE Top 25* (MITRE / SANS) — Debilidades de software más peligrosas

Instituto Tecnológico de Buenos Aires (ITBA)

Documento elaborado a partir de las transcripciones de la clase teórica (Prof. Pablo Eduardo Abad) y de la clase práctica, junto con las diapositivas oficiales de ambas.

Índice

1. Introducción y contexto	2
2. El problema: control de acceso vs. flujo de información	2
3. El problema del confinamiento	3
3.1. Modelo ideal: aislación total	3
4. Canales encubiertos (Covert Channels)	4
4.1. Canales de temporización (Timing Channels)	4
4.2. Canales de almacenamiento (Storage Channels)	5
4.3. Ejemplos de canales encubiertos	5
5. Ataques de canal lateral (Side-Channel Attacks)	5
5.1. Ejemplos	6
6. Control de flujo	6
6.1. Control de flujo en tiempo de compilación	7
6.1.1. Entropía como medida de información	7
6.1.2. Definición formal de flujo de información	7
6.1.3. Flujo directo (ejercicio de la slide)	8
6.1.4. Flujo indirecto	8
6.1.5. Límites	8
6.2. Control de flujo en tiempo de ejecución: aislamiento	9
6.2.1. Virtualización (Virtual Machines)	9
6.2.2. Sandboxing	9
6.2.3. Contenedores	10
7. Malware	10
7.1. Virus / Worm	11
7.2. Troyano	11
7.3. Rootkit	11
7.4. Spyware	12
7.5. Ransomware	12
7.6. Botnet	12
7.7. Ciclo de ataque del malware	12
8. Práctica: CWE Top 25 — Errores de software más peligrosos	14
8.1. Metodología del ranking	14
8.2. Evolución de la clasificación	14
8.3. El CWE Top 25 — 2025, agrupado por categoría	14
8.3.1. Neutralización inadecuada	15
8.3.2. Control inapropiado de un recurso	15
8.3.3. Control de acceso inapropiado	16
8.3.4. Falla en mecanismos de protección	16
8.3.5. Otros (cálculos, condiciones excepcionales, flujo de control)	17
8.4. Debilidades recurrentes (entran y salen del ranking)	17
8.5. Ejemplo: impacto y fase de un buffer overflow	17
Lectura recomendada	18

1 Introducción y contexto

Esta clase cierra el bloque de **Seguridad en aplicaciones** con dos temas: en la parte teórica, **flujo de información** (su última pata: confinamiento, canales encubiertos y control de flujo) y **malware**; en la práctica, los **errores de software más peligrosos** catalogados por el *CWE Top 25* de MITRE/SANS.

Hilo conductor de las clases previas. “Las clases anteriores estuvimos viendo distintos temas de seguridad en aplicaciones desde la parte más teórica de políticas y modelos, hasta problemas más concretos como autenticación y control de acceso. Vamos a retomar un problema que vimos de manera informal al principio: el de *flujo de información*.”

El **flujo de información** ya se introdujo en la Clase 8 (junto con los principios de diseño seguro y las vulnerabilidades). Esta clase lo *profundiza* con el tratamiento formal —entropía de Shannon y seguimiento del flujo— y le agrega lo nuevo: **canales encubiertos**, **ataques de canal lateral**, **técnicas de aislamiento** y, finalmente, la familia completa del **malware**.

Tríada CIA — el marco de referencia

Todo lo que sigue (flujo de información, malware, errores de software) se evalúa por su impacto sobre las tres propiedades clásicas:

- **Confidencialidad:** protege las *lecturas*.
- **Integridad:** protege las *modificaciones*.
- **Disponibilidad:** que quien deba acceder pueda hacerlo cuando lo necesita.

La práctica de esta clase usa explícitamente la tríada para clasificar el *impacto* de cada debilidad de software.

2 El problema: control de acceso vs. flujo de información

El problema. Una política de seguridad dice: “*Un empleado no puede ver los sueldos de otros empleados.*” ¿Cómo podemos implementar esto?

El control de acceso parece la respuesta inmediata (es un problema de confidencialidad). Pero el control de acceso **limita el acceso a operaciones sobre objetos**, no el destino de la información una vez leída. Como los objetos *contienen* información, limitar el acceso limita la información. . . *mientras la información sea estática.*

El control de acceso no impide copiar la información

La información **no es estática**: es actualizada y **puede copiarse**. Un usuario con permiso de *leer* su sueldo podría escribirlo en otro lugar (un archivo de notas) que otros usuarios sí pueden leer. El control de acceso, sentencia a sentencia, no detecta ese **camino de lecturas y escrituras**.

Políticas y control de acceso

Las políticas, por lo general, **restringen el flujo de información**, no el acceso a objetos. Comparar:

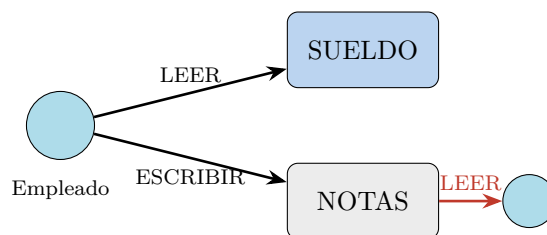


Figura 1: La política se respeta sentencia a sentencia, pero la información *fluye* de SUELDO hacia un tercero vía NOTAS. (*Recreación de la diapositiva 3.*)

Evitar que un empleado *sepa* el sueldo de otro vs. Evitar que un empleado *acceda* a la base de datos de sueldos

¿Entonces los controles de acceso no sirven? Sí sirven, pero en términos formales son **mecanismos abiertos**: aseguran una *parte* del problema, no todo. **Deben ser complementados.**

La metáfora de las capas (aporte de la clase)

Como los mecanismos son abiertos, las empresas piensan la seguridad como **capas de cebolla** (o, como surgió en clase, el “*modelo del queso suizo*”): cada capa tiene agujeros, pero al **superponer muchas** se espera que ningún agujero quede alineado de punta a punta, y cada capa tape los huecos de la anterior.

3 El problema del confinamiento

Confinamiento

El **problema del confinamiento (de acceso)** se refiere a la dificultad de asegurar que un programa o proceso (*sujeto*) **no transmita información** a otra entidad (*objeto*) a la que no está autorizado a acceder, ya sea de manera **directa** o **indirecta**.

Si se pudiera **confinar el flujo de información dentro de un ambiente controlado**, no habría riesgos de confidencialidad ni de integridad. El modelo **Bell–LaPadula** (visto en la Clase 6) define justamente un modelo de seguridad para asegurar el confinamiento de la confidencialidad: en un ambiente totalmente cerrado donde se aplica el 100 % del modelo, da garantías *respecto del flujo*, no solo del lugar donde está la información.

3.1 Modelo ideal: aislación total

El modelo ideal de confinamiento es la **aislación total**. Sus requisitos son simples:

1. Un proceso **no puede comunicarse** con ningún otro proceso.
2. Un proceso **no puede ser observado** por nadie.

Consecuencia: el proceso no revela información. “El sistema más seguro del mundo corre en una computadora apagada, metida en una caja fuerte, enterrada en el fondo del océano, sin que nadie sepa dónde está.”

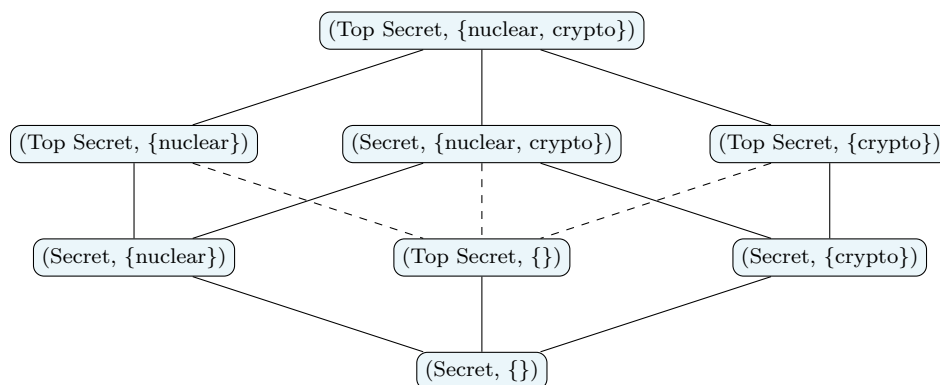


Figura 2: Reticulado de etiquetas de seguridad (nivel, categorías) de Bell–LaPadula que ordena el flujo permitido. (*Recreación de la diapositiva 6.*)

El problema de la aislación total

La aislación total **le quita utilidad al sistema**: no le podemos dar entrada ni observar la salida. Y, en la práctica, lo más difícil es el *segundo* requisito: **cualquier cosa que corra en una computadora tiene medidas observables**, porque usa recursos:

Memoria	Ciclos de CPU
Espacio en disco	Ancho de banda

Observando esos recursos se puede **extraer información** del proceso y de su resultado.

4 Canales encubiertos (Covert Channels)

Canal encubierto (covert channel)

Un **canal encubierto** es un método de comunicación usado para transferir información de manera **oculta**, aprovechando **vías no diseñadas para transmitir datos**. En el contexto de seguridad son un problema porque, al no haber sido pensados para transmitir, es muy poco probable que tengan los **controles** que sí tienen los canales legítimos: sirven para **evadir los controles de seguridad y transmitir datos sin detección**.

4.1 Canales de temporización (Timing Channels)

Codifican datos en la **variación del tiempo entre eventos**. Se monitorea el canal a intervalos regulares y, según cómo cambie la lectura, se infiere información. Ejemplos:

- Latencia de red, carga de CPU, tiempos de respuesta de aplicaciones.

Traffic shaping (aporte de la clase)

Sin entender el *contenido*, contando la cantidad de tramas que circulan se puede inferir *cuándo* hay comunicación y hasta *qué tipo* (un *request/response*, un *streaming* multimedia). Los proveedores de Internet lo usan para **traffic shaping**: analizan la *forma* de la trama y aplican *throttling* (p. ej. a peer-to-peer o video) cuando se excede el ancho de banda. Son aplicaciones “grises”: no maliciosas, pero la misma técnica permite *extraer información*.

4.2 Canales de almacenamiento (Storage Channels)

Usan el **almacenamiento de información en áreas no convencionales o no reguladas** del sistema:

- **Espacio en archivo:** bits no usados / espacio de relleno (*padding*).
- **Atributos de archivos:** metadata como fecha de creación, modificación o acceso.
- **Bits no usados en estructuras de datos:** bits reservados, o lugares de protocolo donde “debe ir un valor aleatorio” y el valor tal vez no sea tan aleatorio.

4.3 Ejemplos de canales encubiertos

- **Exfiltración de información codificando los sonidos del proceso de impresión** (un micrófono de alta fidelidad reconstruye lo que se imprime). dl.acm.org/doi/10.1145/3510583
- **ICMP & DNS Tunneling.** [Akamai — introduction to DNS data exfiltration](#)
- **Emanaciones electromagnéticas.**

DNS tunneling explicado (aporte de la clase)

Muchas empresas filtran el tráfico saliente con *firewalls*, pero **casi todo Internet necesita DNS** para resolver nombres, y durante años los firewalls dejaban pasar las consultas DNS sin control. La idea: si controlás un dominio y su servidor DNS, desde la máquina víctima hacés búsquedas de **subdominios** cuyo nombre es —p. ej.— un bloque del dato a exfiltrar codificado en Base64. Cada consulta llega a tu servidor DNS y, juntando los nombres de los subdominios, **se rearma información arbitraria**, evadiendo los controles. Los canales encubiertos creativos son “bastante complicados de evitar”.

5 Ataques de canal lateral (Side-Channel Attacks)

Ataque de canal lateral

Un **ataque de canal lateral** explota las **emisiones físicas y temporales** de un sistema para extraer información sensible. **No** se basa en vulnerabilidades clásicas del software ni en errores de autenticación, sino en la **observación de efectos colaterales del hardware** durante su funcionamiento. Es especialmente relevante en criptografía: midiendo **tiempos de ejecución, consumo de energía, radiación electromagnética o ruido acústico** se pueden inferir **claves de cifrado**.

Por qué es devastador en criptografía

Toda la seguridad montada sobre criptografía depende de un eslabón: la **clave**. Si un ataque de canal lateral recupera la clave, “se cae toda la seguridad como un castillo de naipes”. Aun **sin éxito total**, basta *sesgar* la probabilidad de algunos bits: una clave de 128 bits con 40 bits sesgados deja al atacante con 2^{88} pruebas por fuerza bruta en lugar de 2^{128} .

5.1 Ejemplos

- **Timing attack:** medir el tiempo de operaciones criptográficas para inferir la clave. (La cantidad de ciclos de reloj para multiplicar/exponenciar números grandes *depende del tamaño* de esos números; combinado con texto plano conocido, se va despejando la incógnita.)
- **Power analysis:** analizar el **consumo de energía** del dispositivo durante las operaciones criptográficas.
- **Emanaciones electromagnéticas:** usar la radiación EM para inferir la operación interna y las claves.

La familia moderna: Rowhammer, Spectre, Meltdown (aporte de la clase)

- **Rowhammer:** golpea (lee/escribe) repetidamente filas de memoria para alterar *por física* bits vecinos sin acceso directo; uno de los más difíciles de evitar.
- **Spectre / Meltdown:** explotan la ejecución especulativa de los procesadores.

Ganaron mucha importancia con los **hyperscalers** (AWS, Google, Azure): en una misma máquina física conviven servidores virtuales de *distintos clientes*. Si desde una VM se pudiera leer la memoria de otra VM en el mismo host, “es una lotería, pero se corre sin parar hasta que sale algo interesante”. Las **smart cards** (transporte, acceso a edificios) son otro blanco clásico: tienen memoria/claves cifradas, pero como están en manos del atacante, históricamente se las rompió midiendo **diferencias de consumo**.

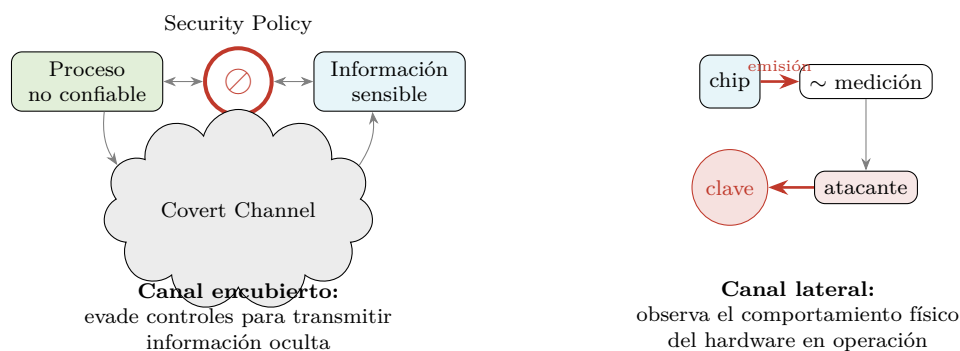


Figura 3: **Canal encubierto** (evadir controles para transmitir) vs. **ataque de canal lateral** (recolectar información por la observación física del hardware). (*Recreación de la diapositiva 16.*)

6 Control de flujo

Hay dos grandes familias de técnicas para hacer cumplir (*enforce*) el control de flujo:

Compile-time vs. run-time enforcement

Tiempo de compilación

Analiza la **propagación de información directamente en el código fuente**. Es **costoso y complejo** ⇒ limitado a componentes muy sensibles.

Tiempo de ejecución

Control dinámico (sandboxing) y **aislación** (virtualización). *Son los métodos más utilizados.*

6.1 Control de flujo en tiempo de compilación

Concepto: cuantificar la información asociada a cada variable, evaluar cómo cambia en cada sentencia, y comparar el valor inicial y final. Si hay una **transferencia que no debiera ocurrir**, fallar.

6.1.1 Entropía como medida de información

Hace casi 90 años, **Claude Shannon** propuso convertir en un solo número la cantidad de información que puede circular por un canal discreto.

Entropía de Shannon

Sea X una variable aleatoria discreta (V.A.D.) que toma valores $x_1, \dots, x_n$:

$$H(X) = - \sum_{i=1}^n p(x_i) \lg p(x_i)$$

(el $\lg$ es el logaritmo en base 2; el resultado se mide en **bits**). **Mide la incertidumbre** a la hora de determinar el valor de la variable: la cantidad de información que *nos falta* conocer para describir completamente el canal.

- **Máximo:** $p(x_i) = 1/n$ (distribución **uniforme**) — no sabemos nada, puede venir cualquier valor.
- **Mínimo** ($= 0$): $p(x_i) = 1$, $p(x_j) = 0$ para $j \neq i$ (**evento único**) — sabemos exactamente qué se transmite, no hay información extra.

Es una función **continua**: cuantifica también los escenarios intermedios, donde conocemos *algo* de la distribución pero no todo.

Entropía condicional

Sean $X (x_1, \dots, x_n)$ e $Y (y_1, \dots, y_m)$ dos V.A.D. Para un valor concreto de Y :

$$H(X | Y = y) = - \sum_{i=1}^n p(x_i | y) \lg p(x_i | y)$$

y, en general, como promedio ponderado de las entropías condicionales:

$$H(X | Y) = \sum_{j=1}^m p(y_j) H(X | Y = y_j).$$

6.1.2 Definición formal de flujo de información

Flujo de información

Sea s el estado de un sistema y t el estado luego de ejecutar comandos $c_1, \dots, c_n$. Sean x_s, y_s valores de objetos en el estado s , e y_t el valor de y en el estado t . **Hay flujo de información de x a y si:**

$$\begin{aligned} H(x_s | y_t) &< H(x_s | y_s) \quad \text{si } y \text{ existe en } s, \\ H(x_s | y_t) &< H(x_s) \quad \text{si } y \text{ no existe en } s. \end{aligned}$$

Es decir: si lo que *desconozco* de x **disminuye** al conocer el estado final de y , entonces hubo transferencia de información de x a y . Si no hubiera transferencia, debería dar **mayor o igual** ($\geq$): el “mayor” aparece, por ejemplo, cuando una asignación *pisa* la variable y destruye información.

En todo programa hay flujo de información —es necesario para funcionar—. Por eso el seguimiento se **ciñe a casos puntuales críticos**, típicamente: verificar que una **función de cifrado no revele información de la clave**.

6.1.3 Flujo directo (ejercicio de la slide)

Ejercicio resuelto — flujo directo

Considerar el comando $y = x + z$, donde $0 \leq x \leq 7$ con igual probabilidad (8 valores) y $Z = \{p(z=1) = 0.5, p(z=2) = 0.25, p(z=3) = 0.25\}$.

Antes del comando (entropía de x , distribución uniforme sobre 8 valores):

$$H(x) = -\sum_{i=1}^8 \frac{1}{8} \lg \frac{1}{8} = -8 \cdot \frac{1}{8} \lg \frac{1}{8} = \lg 8 = \boxed{3}.$$

Conociendo y luego del comando: x solo puede tomar 3 valores ($y-1, y-2, y-3$), con las probabilidades heredadas de z :

$$H(x | y) = -\frac{1}{2} \lg \frac{1}{2} - 2 \cdot \frac{1}{4} \lg \frac{1}{4} = \frac{1}{2} + 2 \cdot \frac{1}{2} = \boxed{1.5}.$$

Como $1.5 < 3$, **hay traspaso de información de x a y** .

6.1.4 Flujo indirecto

La información puede fluir sin que las variables aparezcan en la misma asignación.

Ejercicio resuelto — flujo indirecto (condicional)

`if (x == 0) y = 1; else y = 0;` (x, y *nunca* aparecen en la misma asignación). Con $x \in \{0, 1\}$ y $p(x=0) = 0.5$:

$$H(x) = -2 \cdot \frac{1}{2} \lg \frac{1}{2} = 1, \quad H(x | y) = 0.$$

Como $0 < 1$, **hay traspaso de información** (observando y se deduce x). Es un **canal espacial**.

Ejercicio resuelto — flujo indirecto por comportamiento (loop)

`while (x == 0) {}` (no existe ninguna asignación). Con $x \in \{0, 1\}$, $p(x=0) = 0.5$. Definimos una variable observable $y = 0$ *si el programa termina*:

$$H(x) = 1, \quad H(x | y) = 0 \quad \Rightarrow \quad \text{hay traspaso.}$$

Observar *si el programa termina o no* (un **canal temporal por comportamiento**) da información indirecta sobre x .

6.1.5 Límites

Estas técnicas comparten los límites de la **verificación formal** de sistemas:

- **No es computable verificar todo programa** (límite teórico; en general los no computables son los que nunca terminan).
- Cuando hay **interactividad** (input externo), los casos **explotan** en cantidad de forma exponencial.
- **Los ciclos son un problema**: se hace *loop unrolling* (desenrollar el ciclo N veces); analizar hasta ~ 10 iteraciones suele bastar porque los problemas estructurales aparecen en pocos ciclos.

6.2 Control de flujo en tiempo de ejecución: aislamiento

Las dos formas de aislar procesos

1. **Virtualización**: simularle a la aplicación un **ambiente de ejecución sintético**, separado del real y aislado por definición.
2. **Sandboxing**: **controlar y modificar las interacciones** del sistema con su entorno para aislarlo.

6.2.1 Virtualización (Virtual Machines)

Las VMs usan **hipervisores** para emular hardware y gestionar distintos sistemas operativos sobre un solo host físico:

- **Bare-metal** (tipo 1): KVM, VMware ESXi, Hyper-V.
- **Hosted** (tipo 2): VirtualBox, VMware Workstation.

Cada VM tiene su **propio kernel** $\Rightarrow$ aislamiento fuerte (ni siquiera comparten CPU lógicamente). Contrapartida: es **pesadísimo**, hay que emular todo el ambiente.

6.2.2 Sandboxing

Analiza las interacciones de un proceso con su ambiente y **las modifica para aislarlo**. Ejemplo de diseño: la **isolación de sitios** de Chromium, donde cada *renderer* (`a.com`, `b.com`, `c.com`) corre en su propio sandbox sobre un único *browser process*.

chroot y arbitraje de E/S (aporte de la clase)

En UNIX **toda entrada/salida es un archivo**. El comando `chroot` lanza un proceso con una **vista cortada del filesystem**: la carpeta que le indiquemos pasa a ser su raíz `/`. Uso clásico: los **instaladores de paquetes** corren los scripts de terceros dentro de un sandbox (una vista limpia con sus `/etc`, `/bin`, etc.); luego se verifica *qué archivos nuevos aparecieron* y, si nada es sospechoso, recién ahí se copian al sistema real. A diferencia de las VMs, en sandboxing los procesos **comparten memoria y CPU** $\Rightarrow$ ahí quedan canales no aislados.

Aislamiento a nivel sistema operativo.

- **Linux**: *namespaces* (aislan procesos, usuarios, red, filesystem...) y *cgroups* (controlan/limitan CPU, memoria y red por grupo de procesos).
- **Windows**: *User Mode Scheduling* (similar a namespaces) y *Job Objects* (gestionan grupos de procesos como una unidad, con límites de recursos y prioridades).

6.2.3 Contenedores

Los **contenedores** aprovechan *namespaces* y *cgroups* (Docker es el ejemplo popular) para aislar aplicaciones: múltiples contenedores comparten el **mismo kernel del host** pero mantienen separación lógica de red, procesos y filesystem. Windows soporta contenedores nativos usando características de Windows o Hyper-V. Eliminan la necesidad de un SO completo por instancia (a diferencia de las VMs), dando casi la misma abstracción pero más liviana.

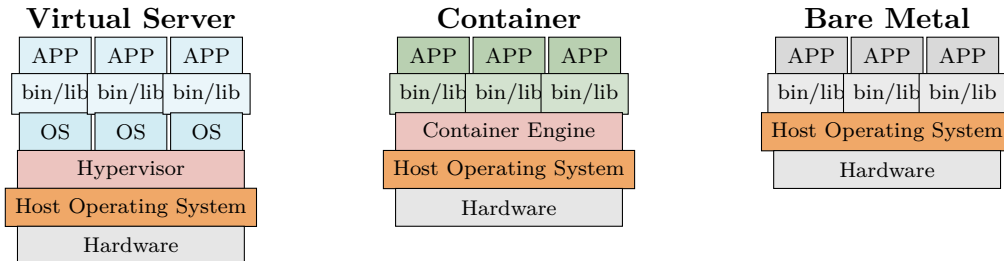


Figura 4: *Stack* comparativo: máquina virtual (cada app con su propio OS sobre el hipervisor), contenedor (apps comparten el kernel del host vía el *container engine*) y *bare metal*. (Recreación de la diapositiva 40.)

7 Malware

Malware (Malicious Software)

Malware es el término con el que se identifica a cualquier programa **diseñado para causar daño, atravesar un control o extraer información**. Un ataque **usualmente usa múltiples tipos de malware** para ejecutar cada parte del ciclo del ataque, y suele **aprovechar vulnerabilidades del software** para lograr su cometido.

El malware se ubica después de las vulnerabilidades en la cadena: hay *amenazas* que se materializan en *vulnerabilidades*, pero hay otra familia —**programas diseñados específicamente con un fin malicioso**—. Lo que les da el nombre distinto no es la técnica (siguen siendo programas), sino que **fueron concebidos exclusivamente con un fin malicioso**.

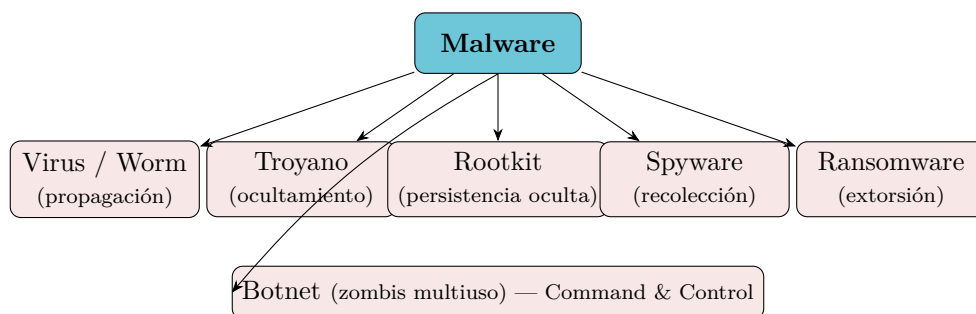


Figura 5: Taxonomía de malware. *Virus/worm* aportan la propagación; *troyano* y *rootkit*, las formas de esconderse; *spyware/ransomware/botnet* describen *qué hace* (el *payload*). Las categorías se combinan.

Propagación vs. payload (aclaración del profe)

Virus y **worm** describen una *forma de propagarse*; **troyano** y **rootkit**, una *forma de esconderse*. Ninguno de los cuatro dice *qué hace* una vez adentro: eso se llama **payload**. Las cosas se combinan: un *virus* se propaga entre máquinas, en cada una se *enquista* como *rootkit*, y su *payload* puede ser un *ransomware*.

7.1 Virus / Worm

- Característica clave: **infectar una máquina y propagarse** usando los medios disponibles. Puede infectar por **explotación de una vulnerabilidad remota** o reescribiendo un **recurso compartido** (archivo ejecutable).
- Un **virus** suele tener dos componentes: **replicación** + **ejecución** (según el componente de ejecución, se le da otro nombre).
- El **worm** se especializa en la **infección por red**, propagándose incluso por email (**Melissa**, 1999). El primer gusano nació por error en un experimento de cómputo distribuido y saturó las máquinas por falta de memoria.
- **SQL Slammer** infectó **75.000 computadoras en 10 minutos** aprovechando bases de datos SQL expuestas a Internet (asociados a *denegación de servicio*).

7.2 Troyano

- **Malware oculto dentro de otro software** que parece benigno y de interés para la víctima (del caballo de Troya). Se usa para **establecer al atacante dentro del equipo infectado**.
- Típico en *cracks* / versiones piratas (Windows, Office) y en videojuegos móviles que empresas compran y modifican para incluir troyanos.
- **PC-Write** (1986): uno de los primeros *shareware* (editor de texto) que contenía un troyano que **borraba todos los archivos** de la máquina.

7.3 Rootkit

- Software diseñado para **ocultar malware** dentro de la máquina infectada, con **contramedidas activas**: ofuscación, encriptación, reescritura y reemplazo de herramientas del sistema para quedar oculto a administradores y antivirus.
- Pueden operar a **nivel kernel o de usuario**. Un rootkit en Linux puede **interceptar system calls** (p. ej. ocultar su proceso de *ps* o sus archivos de */proc*). Los más nuevos se meten en el **firmware/BIOS** (de ahí nacen *Secure Boot* y el boot firmado), persistiendo aunque se formatee el disco.
- **NTRootkit** (1999): primer rootkit para Windows NT. **Stuxnet** (2010): usó rootkit para recolectar información por *meses* antes de lanzar el ataque que afectó las plantas nucleares de Irán (atribuido a servicios de inteligencia; las máquinas estaban aisladas de Internet).

Por qué los rootkits son tan peligrosos

Trabajan “un nivel atrás”: lees memoria y no hay nada, lees disco y no hay nada. Cuanto más sofisticado, de más cosas se esconde ⇒ los más avanzados **no son detectables por antivirus**. Contrapartida: por trabajar a tan bajo nivel, una actualización del SO puede romperlos.

7.4 Spyware

- Software diseñado para **recolectar información** del usuario infectado, manteniendo el **perfil más bajo posible**.
- Se especializa en **ocultarse** y usualmente establece alguna forma de **covert channel (túnel)** para exfiltrar; en el mercado hogareño (sin equipo de seguridad) suele usar canales normales.
- Algunas empresas lo desarrollan **a pedido de gobiernos** para vigilancia digital o espionaje (caso mencionado: *Palantir*).

7.5 Ransomware

- Software para **bloquear el acceso** a la información, pidiendo **dinero** (rescate) para restaurarla. Habilitado por las **criptomonedas** (mecanismo de cobro descentralizado y difícil de rastrear).
- Variantes: cifrar la información y pedir rescate por la clave; o **exfiltrarla** y pedir rescate para *no publicarla*.
- **CryptoLocker** (2013): enviado vía mail, infectaba al abrir el correo; se estima ~USD 3M robados ese año. **WannaCry** (2017): aprovechó una vulnerabilidad conocida de Windows para **distribuirse en la red local**; >200K máquinas, costo estimado USD 4B. Es **pesadilla en el ámbito empresarial**: un servidor o base de datos comprometido pone en riesgo activos muy importantes.

7.6 Botnet

- Software para desplegar **bots / agentes multiuso** que quedan a la **escucha de instrucciones (máquinas zombi)**. Se conectan a un servidor de **Command & Control (C&C)** y preguntan “qué tengo que hacer”.
- Usos: **ataques distribuidos (DDoS)**, minería de cripto, accesos anónimos, **distribuir spam**, robar credenciales bancarias, usarlas de *pivot* para otro ataque (cuando se rastrea, se llega a la zombi y no al original).
- Las infraestructuras C&C son **sistemas distribuidos descentralizados, sin punto único de falla**, difíciles de dar de baja. **Srizbi** (2008): la botnet más grande documentada; responsable de ~la **mitad del spam mundial** (~60B mails/día).

La “economía gris” (aporte de la clase)

Servicios no estrictamente ilegales que requieren “algo no muy santo” por detrás: envío de spam evadiendo filtros de Gmail (vía computadoras hogareñas infectadas), o “cómputo distribuido en alquiler” que en realidad son botnets. Ejemplo moderno: servicios que ofrecen acceso a las APIs de Anthropic/OpenAI a un cuarto del precio, presumiblemente usando **API keys robadas** con estos mecanismos de propagación + spyware.

7.7 Ciclo de ataque del malware

El ciclo de un malware suele tener, esquemáticamente, estas fases. Lo interesante es que **toda la parte intermedia** (desde el reconocimiento hasta el bucle de escalamiento) es **genérica** respecto del payload final $\Rightarrow$ hoy se venden **malware kits** que “productizan” esa parte (aunque ser genérico también los hace más detectables, y hubo kits que eran a su vez troyanos contra quien los usaba).

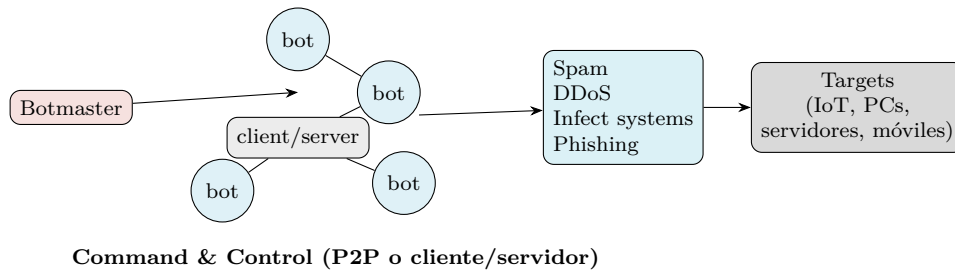


Figura 6: Arquitectura de una botnet: el *botmaster* comanda nodos C&C (P2P o cliente/servidor) que ejecutan actividades maliciosas sobre los *targets*. (Recreación de la diapositiva 42.)

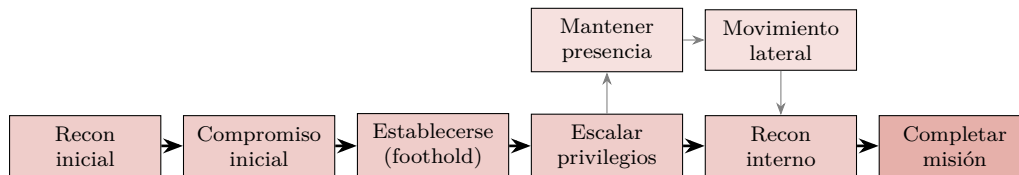


Figura 7: Ciclo de ataque del malware. El lazo *escalar* → *mantener presencia* → *movimiento lateral* → *recon interno* se repite varias veces. (Recreación de las diapositivas 43–52.)

Fase	Qué ocurre
Reconocimiento inicial	Se identifica a la víctima y se recopila información: responsables, puntos débiles, intereses; se mapean redes sociales y se establecen relaciones falsas.
Compromiso inicial	El malware se “ <i>weaponiza</i> ” (empaqueta para ser ejecutado por la víctima); con un ataque dirigido se compromete una o más máquinas. Puede durar varios días.
Establecerse (foothold)	Establece comunicación con su C&C , asegura persistencia y almacenamiento; baja y compone nuevas herramientas. Suele durar minutos.
Escalar privilegios	Eleva los privilegios obtenidos usando vulnerabilidades locales/remotas. Suele durar minutos.
Recon interno	Analiza comunicaciones, mapea otras máquinas/usuarios de interés; del descubrimiento baja y compone nuevos malwares.
Movimiento lateral	Explotación remota de vulnerabilidades para incrementar la base infectada.
Mantener la presencia	Asegura persistir en las nuevas máquinas infectadas.
Completar misión	Ejecuta el payload : extrae/acumula información y la envía en horarios de menor monitoreo; en ransomware, infecta el máximo de máquinas antes de activar el bloqueo.

8 Práctica: CWE Top 25 — Errores de software más peligrosos

De qué va la práctica

La parte práctica trabaja sobre el **CWE Top 25** (*Common Weakness Enumeration*) de MITRE/SANS: el ranking de las **25 debilidades de software más peligrosas**. Son debilidades **fáciles de encontrar y explotar**, que pueden permitir al adversario **controlar el sistema, robar datos o impedir el funcionamiento**.

8.1 Metodología del ranking

- El **puntaje** se calcula principalmente en función de la **frecuencia** de aparición y del **impacto/daño**. No es lo mismo algo muy frecuente pero de bajo impacto, que algo frecuente y de alto impacto.
- Se publica **anualmente desde ~2010**. Cada año varias organizaciones reportan los problemas observados; MITRE los cataloga y, procesando ~un año de datos, produce el ranking. **Fluctúa**: hay errores que entran, salen y **vuelven a aparecer**.
- Por cada error, la página (cwe.mitre.org/top25) detalla: **fase del ciclo de vida** donde se origina (requisitos / arquitectura y diseño / implementación), **lenguajes** donde aparece más, **consecuencias** (impacto en la tríada CIA), **probabilidad de explotación**, ejemplos y **casos reales observados**.

Recomendación de lectura. Conviene leer los errores **agrupados por categoría**, no en el orden del ranking: errores de la misma categoría tienen **soluciones similares**, así que entender uno ayuda a entender los demás (“si entendés SQL injection, XSS es parecido”).

8.2 Evolución de la clasificación

2011 — 3 grandes grupos	2025 — “Pillars” (más desglosado)
• Interacción insegura entre componentes • Manejo riesgoso de recursos • Defensas abiertas / porosas	<i>Improper Neutralization; Improper Control of a Resource Through its Lifetime; Protection Mechanism Failure; Improper Access Control; Improper Adherence to Coding Standards; Incorrect Calculation; Improper Interaction Between Multiple Correctly Behaving Entities; Insufficient Control Flow Management; Incorrect Comparison; Improper Check or handling of exceptional conditions.</i>

Los *pillars* de 2025 terminan “cayendo” en los tres grupos originales (más el cuarto). Las categorías **más frecuentes** son la 1ª, la 2ª y la 4ª: **neutralización inadecuada, control inapropiado de recursos y control de acceso inapropiado**. (Referencias: cwe.mitre.org/data/definitions/1000.html.)

8.3 El CWE Top 25 — 2025, agrupado por categoría

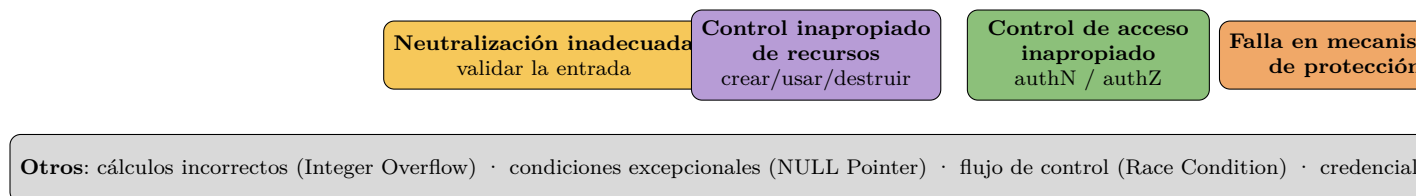


Figura 8: Pilares/categorías del CWE Top 25. Las tres primeras concentran la mayoría de las debilidades del ranking. (*Recreación de las diapositivas prácticas 3–5.*)

8.3.1 Neutralización inadecuada

Idea y mitigaciones

Idea: asegurarse de que el *input/output* tenga ciertas propiedades antes de usarse. No se valida correctamente la entrada y con ella se construye un comando (SQL, sistema operativo, HTML...). **Mitigaciones comunes** (las mismas para toda la categoría): **formatear la entrada**, **listas blancas / listas negras** (lo permitido / lo prohibido) y **validación de la entrada**.

#	Debilidad	Rank
1	Cross-site Scripting (XSS)	1
2	SQL Injection	2
3	OS Command Injection	9
4	Code Injection	10
5	Improper Input Validation	18
6	Command Injection	23

8.3.2 Control inapropiado de un recurso

Idea

La creación, uso y destrucción de un recurso debe hacerse correctamente. Muchos tienen que ver con **buffer overflow** y manejo de memoria (más ligado a lenguajes de bajo nivel como C); este año aparecen muchísimos. Se distingue el *classic buffer overflow* (no se chequea el tamaño de la entrada) del que se produce, p. ej., por llamadas recursivas sucesivas (stack vs. heap). *Path traversal*: armar un *path* que termina dando acceso a cosas que no se deberían tener. *Unrestricted upload*: no controlar un archivo subido (un PNG que en realidad es un ejecutable). *Deserialization*: convertir una cadena/estructura sin control.

#	Debilidad	Rank
1	Out-of-bounds Write (escritura fuera de límites)	5
2	Path Traversal	6
3	Use After Free	7
4	Out-of-bounds Read (lectura fuera de límites)	8
5	Buffer Copy without Checking Size of Input (Classic Buffer Overflow)	11
6	Unrestricted Upload of File with Dangerous Type	12
7	Stack-based Buffer Overflow	14
8	Deserialization of Untrusted Data	15
9	Heap-based Buffer Overflow	16
10	Exposure of Sensitive Information to an Unauthorized Actor	20
11	Server-Side Request Forgery (SSRF)	22
12	Allocation of Resources Without Limits or Throttling	25

8.3.3 Control de acceso inapropiado

Idea

Autenticación, autorización y registro. Aparecen autorizaciones/autenticaciones incorrectas o **faltantes**, y formas de *saltear* la autorización. La *authorization* a secas (*Incorrect Authorization*) “engloba todo”. Mitigación: el control de acceso depende mucho de la **arquitectura de diseño** (dónde poner el control) y de la **implementación** (no configurarlo mal); no suele depender de un lenguaje específico.

#	Debilidad	Rank
1	Missing Authorization (autorización faltante)	4
2	Improper Access Control (control de acceso inapropiado)	19
3	Incorrect Authorization	17
4	Missing Authentication for Critical Function	21
5	Authorization Bypass Through User-Controlled Key	24

8.3.4 Falla en mecanismos de protección

Idea

Mal uso / no uso de mecanismos de protección. Incluye el **uso incorrecto de algoritmos criptográficos** (o no usar estándares). “No tenés que inventar tu función de encriptación; tenés que usar los estándares y usarlos correctamente.”

#	Debilidad	Rank
1	Cross-Site Request Forgery (CSRF)	3

8.3.5 Otros (cálculos, condiciones excepcionales, flujo de control)

Pilar	Debilidad	Rank
Cálculos incorrectos	Integer Overflow or Wrap-around	30
Condiciones excepcionales (estándares de código)	NULL Pointer Dereference	13
Manejo insuficiente del flujo de control	Race Condition	36

Estos aparecen en el *on-the-cusp list* (puestos ~26–36): debilidades que “están ahí, queriendo entrar y salir”. Cada una indica su posición del año anterior y cuánto subió/bajó. (Ref: cwe.mitre.org/top25/archive/2025/2025_onthecusp_list.html.)

8.4 Debilidades recurrentes (entran y salen del ranking)

“Siempre vuelven”

Algunas debilidades desaparecen un tiempo y **vuelven a aparecer**:

- **Credenciales hardcoded** (codificadas en el código): hasta el año pasado siempre estaban en el ranking.
- **Exposure of Sensitive Information** (exponer información sensible): p. ej. un mensaje de error que filtra parte del directorio o de la base de datos. “Había desaparecido y volvió.”
- **Uso incorrecto / ausencia de criptografía estándar**: estuvo entre los primeros en su momento; “menos mal que ya no aparece”.

Clásicos que nunca desaparecen: *Cross-site Scripting* (rank 1, también n°1 el año pasado) y *SQL Injection* (rank 2, estaba en el 3 el año anterior); fluctúan en los primeros puestos. La aparición masiva de **buffer overflows** este año (4–5 entradas) llamó la atención: a más uso de un lenguaje/registro de datos, más información ⇒ más detección.

8.5 Ejemplo: impacto y fase de un buffer overflow

Lectura de una ficha CWE (caso de la clase)

Para **Buffer Overflow**, la ficha muestra:

- **Impacto (tríada CIA)**: *Integridad* (puede ejecutar comandos no autorizados y modificar datos), *Disponibilidad* (puede producir *DoS* / crash) y *Confidencialidad* (si se leen datos confidenciales).
- **Fase donde se origina**: *requisitos* (elegir lenguaje/compilador adecuado), *arquitectura y diseño* (qué librerías/frameworks usar), *implementación*.
- **Probabilidad de explotación**: alta. Más abajo: relaciones con otros errores, lenguajes frecuentes, ejemplos y casos reales observados, y métodos de detección.

Guía de ejercicios. La práctica selecciona ejercicios de los tipos más comunes —XSS, *Race Condition*, *Web sessions*— pero conviene leer todas las fichas **agrupadas por categoría** (es más fácil de entender y recordar). Algunos nombres de ataques de canal lateral (p. ej. *RAMBO*: variaciones electromagnéticas que forman radiofrecuencias captables desde afuera) también aparecen en la guía, conectando con la parte teórica.

Lectura recomendada

Capítulos 16-1 y 17

Computer Security: Art and Science

Matt Bishop

Recursos de la práctica: CWE Top 25 de MITRE (cwe.mitre.org/top25), su jerarquía de *Research Concepts* (cwe.mitre.org/data/definitions/1000.html) y OWASP como referencia complementaria de seguridad en aplicaciones web.